## Create a Module

You're ready to write code in your Mix project. We'll start with a simple module containing a few functions. Create the following file in lib/examples/modules/one.ex:

elixir_examples/lib/examples/modules/one.ex

```
defmodule Examples.Modules.One do
  def hello do
    :world
  end

  def welcome(name) do
    "Welcome #{name}"
  end
end
```

The code inside of the defmodule block becomes the module once Elixir compiles it. defmodule is a special type of syntax called a macro.

Similarly, def is a macro that defines a function. Functions can have arguments (like welcome/1 does) or they can exclude arguments (like hello/0).

These functions are inside of a module and are named, so we call them named functions. We refer to named functions with the syntax Module.function/arity, like Examples.Modules.One.welcome/1. (Arity refers to the number of arguments passed into a function.) Sometimes the module name is excluded if it's referring to the current module that you're in.

**Macros: With Power Comes Responsibility**



The best advice about macros is "don't write macros," as Chris McCord wrote in *Metaprogramming Elixir [McC15]*.

This is slightly sarcastic advice, but it highlights the fact macros can be tricky to write. Especially when you're first starting out.

Macros are functions that emit code when compiled. Macros let you quickly integrate complex libraries or functions into your code. They are most often used by library authors or by developers who are creating an abstraction. This is referred to as metaprogramming.

We won't cover macros in depth in this book, although they will pop up occasionally. Chris McCord's book is my go-to resource for macros, and the official documentation is also very good.

It's not required to make your module names match the file name, but the convention is to do so because it helps with organization. Every .ex file inside